# VENDING MACHINE CONTROLLER - TECHNICAL DOCUMENTATION

By:Ravi Patel

**TABLE OF CONTENTS**

1. Introduction
2. Why Pulse Detection is Critical
3. How to Calculate Register Sizes
4. Overflow Protection Mechanism
5. Timeout Logic & Real-World Timing
6. Complete Design Summary
7. Inputs, Outputs, and Internal Signals
8. PRODUCT QUANTITY ENCODING & DISPENSE COMPLETION SIGNAL

## 1. INTRODUCTION

This document provides a comprehensive technical explanation of a Verilog-based vending machine controller. Unlike typical documentation that merely describes what the code does, this report focuses on the **WHY** and **HOW** of critical design decisions, particularly:

- Why pulse detection prevents catastrophic counting errors
- How register bit widths are mathematically calculated
- Why overflow protection is necessary and how it works
- How timeout values relate to real-world user behavior

The design is built around a **Finite State Machine (FSM)** that ensures deterministic, safe operation for a commercial vending machine application.

**DESIGN OVERVIEW**

| Phase | Description |
|---|---|
| Selection | User selects one or more products; total price is calculated |
| Collection | Accepts coins/bills until sufficient money is inserted |
| Verification | Checks if entered amount >= total; signals ready state |

| Phase | Description |
| --- | --- |
| Dispensing | Dispenses product and returns correct change to user |

---

**2. WHY PULSE DETECTION IS CRITICAL (EDGE DETECTION)**

**THE PROBLEM: Button Bounce & Duration**

When a user presses a physical button, it does not create a clean, single-clock-cycle signal. Instead, the button signal remains HIGH for the entire duration the button is pressed - which can be millions of clock cycles.

**Example Calculation:**

| Parameter | Value |
| --- | --- |
| Button press time | 0.2 seconds |
| System clock freq | 50 MHz (50,000,000 cycles/sec) |
| Button HIGH cycles | 0.2 × 50,000,000 = 10,000,000 cycles |

Without pulse detection, a single button press would trigger the selection logic **10 MILLION times!**

**Code WITHOUT pulse detection (WRONG):**

```
if (selection)
   total <= total + price_LUT[product_id];
// This executes 10,000,000 times!
// A $10 item becomes $100,000,000!
```

**THE SOLUTION: Rising Edge Detection**

The solution is to detect only the **rising edge** of the signal. This ensures the action occurs exactly once per button press.

**Code WITH pulse detection (CORRECT):**

```
// Store previous state
always @(posedge clk) begin
   select_d <= selection;
end
// Detect rising edge
wire select_pulse = selection & ~select_d;
```

**Step-by-Step Example:**

| Clock | selection | select_d | ~select_d | select_pulse | Result |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 0 | No trigger |
| 2 | 1 | 0 | 1 | 1 | ✓ Triggered once |
| 3 | 1 | 1 | 0 | 0 | No trigger |
| 4 | 1 | 1 | 0 | 0 | No trigger |
| 5-10M | 1 | 1 | 0 | 0 | No trigger |

**Result:** The button press is counted exactly **once**, regardless of duration.

---

**3. HOW TO CALCULATE REGISTER SIZES**

**THE GOLDEN RULE OF BINARY REPRESENTATION**

- Every register must be sized correctly to prevent overflow.
- n bits can represent **2^n unique values (0 to 2^n - 1)**
- To find the required bits:

2^n ≥ Maximum Value + 1

**COMMON BIT WIDTH EXAMPLES**

| Bits (n) | 2^n | Max Value | Use Case in Design |
|---|---|---|---|
| 1 | 2 | 1 | Flags (ready, dispensed) |
| 4 | 16 | 15 | product_id (16 products) |
| 7 | 128 | 127 | money_value, price_LUT |
| 8 | 256 | 255 | TOO SMALL for this design! |
| 9 | 512 | 511 | entered_money, total, change |
| 26 | 67,108,864 | 67,108,863 | sec_cnt (count to 50M) |

**DETAILED CALCULATION EXAMPLES**

**Example 1: product_id (4 bits)**

- Requirement: 16 products (0-15)
- 2^n ≥ 16 → 2^4 = 16 ✓
- Answer: **4 bits**

**Example 2: money_value (7 bits)**

- Requirement: Largest coin/bill = 100
- 2^7 = 128 ✓ sufficient
- Answer: **7 bits**

**Example 3: entered_money (9 bits) – CRITICAL**

- Requirement: Multiple expensive items
- Worst-case: 5 × 100 = 500 units
- 2^9 = 512 ✓ sufficient
- Answer: **9 bits**

**Why 8 bits would fail:**

```
reg [7:0] entered_money;  // WRONG!
entered_money = 250;
entered_money = entered_money + 20;  // = 270 OVERFLOW!
```

**Example 4: sec_cnt (26 bits)**

- Requirement: 1-second timer at 50 MHz
- 2^26 = 67,108,864 ✓ sufficient
- Answer: **26 bits**

---

**4. OVERFLOW PROTECTION MECHANISM**

**WHY extra_money IS NECESSARY**

Even with correct register sizes, inserting more than **MAX_LIMIT** can overflow:

| Condition | Value |
|---|---|
| MAX_LIMIT | 511 |
| entered_money | 500 |
| User inserts | 100 → attempted total 600 > 511 ⚠️ |

**TWO-REGISTER SOLUTION**

- **entered_money [9 bits]** – stores up to MAX_LIMIT

- **extra_money [7 bits]** – stores overflow

code:

```
if (entered_money + money_value <= MAX_LIMIT)
   entered_money <= entered_money + money_value;
else begin
   extra_money   <= (entered_money + money_value) - MAX_LIMIT;
   entered_money <= MAX_LIMIT;
end
```

**Overflow Handling Table:**

| Step | Action | entered_money | extra_money |
|---|---|---|---|
| 1 | Initial state | 500 | 0 |
| 2 | User inserts 100 | 600 | 0 |
| 3 | Check overflow | YES | - |
| 4 | Calculate extra | 511 | 89 |
| 5 | Final result | 511 ✓ | 89 ✓ |

**Why extra_money = 7 bits:**

- Max overflow = 127 → 2^7 = 128 ✓

---

**5. TIMEOUT LOGIC & REAL-WORLD TIMING**

**THE PROBLEM: Abandoned Transactions**

Users may:

- Walk away mid-transaction
- Insert partial payment and leave
- Get distracted

**Solution:** 2-minute timeout to prevent machine lockup

**1-SECOND TIMER**

```
reg [25:0] sec_cnt;  // 26-bit counter
reg tick_1s;         // 1-second pulse
```

```
always @(posedge clk) begin
   if (sec_cnt == 50_000_000 - 1) begin
      sec_cnt <= 0;
      tick_1s <= 1;
   end else begin
      sec_cnt <= sec_cnt + 1;
      tick_1s <= 0;
   end
end
```

**Idle Counter:**

```
reg [6:0] idle_count;
localparam TIMEOUT_LIMIT = 7'd120;

always @(posedge clk) begin
   if (rst || state == IDLE)
      idle_count <= 0;
   else if (tick_1s)
      idle_count <= idle_count + 1;
end
```

- **idle_count = 7 bits** → counts 0-127
- **TIMEOUT_LIMIT = 120 sec** → balances convenience & availability

**Timeout Action:**

```
if (cancel_pulse || idle_count >= TIMEOUT_LIMIT) begin
   change_out <= entered_money + extra_money;
   state <= CLEAR;
end
```

idle_count resets when:

• state == IDLE

• money_pulse

• select_pulse

• cancel_pulse

• disp_pulse

---

**6. COMPLETE DESIGN SUMMARY**

**All Register Sizes:**

| Register | Bits | Max | Rationale |
|---|---|---|---|
| product_id | 3 | 8 | 8 products (0-15) |
| money_value | 7 | 127 | Largest bill = 100 |
| price_LUT[i] | 7 | 127 | Max product price = 100 |
| entered_money | 9 | 511 | 5 × 100 = 500 |
| extra_money | 7 | 127 | Max overflow = 127 |
| total | 9 | 511 | Sum of multiple products |
| change_out | 9 | 511 | Matches entered_money |
| idle_count | 7 | 127 | Counts to 120 sec |
| sec_cnt | 26 | 67M | Counts 50M cycles |

**KEY DESIGN PRINCIPLES**

1. **Pulse Detection** – ensures single trigger per button press
2. **Safety-First Sizing** – oversize registers to avoid overflow
3. **Overflow Protection** – dual-register approach
4. **Deterministic FSM** – clear state transitions
5. **Real-World Timing** – 2-minute timeout balances usability
6. **Money Safety** – all funds accounted for

**CONCLUSION**

- Register sizing, edge detection, and defensive programming prevent catastrophic failures
- Every design choice is mathematically justified for robust operation

---

**7. INPUTS, OUTPUTS, AND INTERNAL SIGNALS**

This section lists **all interface ports, internal signals, their sizes, and functionality** in the vending machine controller. It serves as a complete reference for system integration and verification.

### 7.1 INPUTS

| Signal Name | Width | Type | Description |
|---|---|---|---|
| **clk** | **1** | **Input** | **System clock (50 MHz)** |
| **rst** | **1** | **Input** | **Global reset** |
| **selection** | **1** | **Input** | **Product selection button** |
| **dispence** | **1** | **Input** | **Dispense confirmation button** |
| **cancel** | **1** | **Input** | **Cancel transaction button** |
| **money_valid** | **1** | **Input** | **Indicates money insertion event** |
| **money_value** | **7** | **Input** | **Inserted coin/bill value** |
| **product_id** | **4** | **Input** | **Selected product ID (0–15)** |

**Notes on inputs:**

- All button signals are edge-detected internally (*_pulse) to prevent multiple triggers
- Coin/bill input uses 7 bits to support up to 127 units

---

### 7.2 OUTPUTS

| Signal Name | Width | Type | Description |
|---|---|---|---|
| ready | 1 | Output | Indicates enough money has been inserted |
| dispenced | 1 | Output | Indicates transaction completed |
| product_delivered | 1 | Output | Indicates product successfully delivered |
| entered_money | 9 | Output | Total inserted money |
| extra_money | 7 | Output | Overflow money beyond MAX_LIMIT |
| total | 9 | Output | Total cost of selected products |
| change_out | 9 | Output | Money returned to user |

---

### 7.3 INTERNAL REGISTERS & SIGNALS

| Signal Name | Width | Type | Description |
|---|---|---|---|
| money_d | 1 | reg | Previous state of money_valid |
| select_d | 1 | reg | Previous state of selection |
| cancel_d | 1 | reg | Previous state of cancel |
| disp_d | 1 | reg | Previous state of dispence |
| money_pulse | 1 | wire | Rising edge detection of money_valid |
| select_pulse | 1 | wire | Rising edge detection of selection |
| cancel_pulse | 1 | wire | Rising edge detection of cancel |
| disp_pulse | 1 | wire | Rising edge detection of dispence |

**Notes on internal signals:**

- select_d and cancel_d are used for rising-edge detection of physical buttons
- entered_money + extra_money always represents **total inserted money**
- sec_cnt generates a 1-second pulse, used to increment idle_count
- idle_count is compared to TIMEOUT_LIMIT (120 sec) for auto-cancel

---

**7.4 SIGNAL FLOW & WORKING**

1. **Button Press Detection**
    - select_btn and cancel_btn → edge-detected → triggers FSM transitions
2. **Money Collection**
    - coin_input added to entered_money
    - Overflow stored in extra_money
3. **Timeout Logic**
    - sec_cnt generates 1s ticks
    - idle_count increments per second
    - If idle_count >= TIMEOUT_LIMIT → FSM clears transaction, outputs all money as change_out
4. **FSM Outputs**
    - dispense HIGH → triggers product delivery
    - ready HIGH → indicates enough money inserted
    - change_out calculated using entered_money + extra_money - total

5. **State Transitions**
    - IDLE → SELECTION → COLLECTION → VERIFICATION → DISPENSE → CLEAR → IDLE
    - Cancel or timeout can interrupt the flow → CLEAR

---

**8. PRODUCT QUANTITY ENCODING & DISPENSE COMPLETION SIGNAL**

**8.1 Product Quantity Encoding Using product_id**

To reduce hardware complexity and avoid additional input signals, the **quantity of the selected product is encoded within the product_id itself**.

The **least significant bit (LSB)** of product_id represents the quantity:

| product_id[0] | Quantity |
|---|---|
| 0 | 1 unit |
| 1 | 2 units |

The remaining upper bits (product_id[3:1]) represent the product index.
This approach enables multi-quantity selection **without increasing pin count** or FSM complexity.

---

**8.2 Quantity-Based Total Price Calculation**

The total amount is calculated by combining the product price and selected quantity.
This ensures correct billing for both single and double quantity selections while maintaining existing overflow protection logic.

---

**8.3 Dispense Completion Signal (dispenced) – Dual-Purpose Role**

The signal **dispenced does NOT only mean "product delivered"**.
Instead, it represents **end-of-process notification** for the entire transaction.

dispenced is asserted in **both final outcomes**:

| Scenario | Meaning of dispenced |
|---|---|
| Successful purchase | Product delivered and transaction completed |
| Cancel / Timeout | Transaction aborted and money returned |

In both cases, dispenced indicates that the vending machine has **finished processing** the user request.

---

**8.4 Practical Use of dispenced in Hardware**

The dispenced signal is intended to drive **external notification hardware**, such as:

- Buzzer sound
- Voice announcement module
- Completion LED

---

### 8.5 Logical vs Physical Completion Signals

The design intentionally separates logical confirmation from physical notification:

| Signal | Purpose |
|---|---|
| product_delivered | Indicates whether a product was actually issued |
| dispenced | Indicates that the transaction process has ended |

This separation ensures:

- Clear distinction between **success** and **termination**
- Safe operation of physical devices
- Accurate status reporting to external controllers

---

### 8.6 End-of-Process Flow Summary

1. User selects product and inserts money
2. FSM evaluates payment and user action
3. One of two outcomes occurs:
    - Product is delivered
    - Transaction is cancelled or times out
4. dispenced is asserted to indicate **process completion**
5. System resets safely to IDLE state

---